Facultad de Ingeniería
Bernard Wand-Polak
Cuareim 1451
11.100 Montevideo, Uruguay
Tel. 2902 15 05 Fax 2908 13 70
www.ort.edu.uy

Obligatorio

Algoritmos y Estructuras de Datos 2 – Programación avanzada

Documentación: estructuras utilizadas y justificación de órdenes

EVALUACIÓN	Obligatorio	**FECHA DE ENTREGA**	09/11/2026
MATERIA	Algoritmos y Estructuras de Datos 2 – Programación avanzada		
CARRERA	Analista en tecnologías de la información	**GRUPO**	MB2
DOCENTE	Fabian Fernandez		

Integrantes del equipo

Nombre	Número de estudiante
Mauricio Prado	331273
Pablo de Melo	330897

Montevideo, noviembre de 2026

Facultad de Ingeniería
Bernard Wand-Polak
Cuareim 1451
11.100 Montevideo, Uruguay
Tel. 2902 15 05 Fax 2908 13 70
www.ort.edu.uy

1. Introducción

En este documento contamos cómo resolvimos el obligatorio: qué estructuras elegimos para cada parte del problema y por qué cada operación cumple el orden que pide la letra. El sistema tiene dos mitades bastante distintas. Por un lado está el registro y la consulta de despliegues de modelos (operaciones 1 a 8), donde lo que importa es que las búsquedas y los listados sean eficientes. Por otro lado está la red de regiones y enlaces (operaciones 9 a 13), que modelamos con un grafo.

En las dos partes la idea fue quedarnos con las herramientas que vimos en clase (el ABB con comparador, el grafo con matriz de adyacencia, el BFS con tuplas y Dijkstra) y adaptarlas al problema, en vez de inventar soluciones nuevas. A lo largo del documento usamos la siguiente notación:

- **n**: cantidad total de despliegues registrados.
- **k**: cantidad de despliegues de una modalidad dada.
- **M**: cantidad máxima de regiones (el parámetro `maxRegiones` de `inicializarSistema`).

2. Estructuras utilizadas

Antes de entrar en detalle, este es el resumen de qué usamos y para qué:

Estructura	Implementación	Se usa en
ABB genérico	`ABB<T extends Comparable<T>>` con comparador opcional	Operaciones 2 a 8 y 11
Despliegue	Clase de dominio, `Comparable` por id	Operaciones 2 a 8
Comparador por código	`ComparadorDesplieguePorCodigo`	Operaciones 2, 6, 7 y 8
Árboles por modalidad	Array de 5 ABB indexado por `Modalidad.getIndice()`	Operaciones 2 y 8
Grafo	Matriz de adyacencia de `Arista` + array de `Region`	Operaciones 1 y 9 a 13
Cola genérica	`ICola<T>` / `Cola<T>` con nodos	Operación 11 (BFS)
Tupla	`Tupla<Q, V>`	Operaciones 11, 12 y 13

2.1 ABB genérico

Toda la parte de despliegues se apoya en un ABB genérico, `ABB<T extends Comparable<T>>`, que armamos a partir del `ABBConComparador` del curso. Se puede construir sin comparador, y en ese caso usa el `compareTo` de `T`, o con un `Comparator<T>`. Esto es lo que nos deja tener varios árboles ordenados por criterios distintos sobre la misma clase. Además de `insertar`, tiene:

- `buscar(x)`: baja por una sola rama y, además del elemento, devuelve cuántos nodos recorrió (en la clase `ResultadoBusqueda<T>`). El nodo encontrado también se cuenta, así que si el elemento está en la raíz devuelve 1.
- `listarAscendente()` y `listarDescendente()`: recorrido in-order (izquierda, nodo, derecha) y su espejo (derecha, nodo, izquierda). Van armando el texto con `|` entre elementos, sin dejar uno

al final.

- `vaciar()`: deja la raíz en `null`; lo usamos en `inicializarSistema`.

Un detalle que nos pareció importante: para saber si encontró el elemento, `buscar` usa `comparar(...) == 0` y no `equals`. Eso es lo que permite buscar en el árbol por código usando un despliegue que solo tiene el código cargado. Si usáramos `equals` (que en `Despliegue` compararía por id), en ese árbol nunca encontraríamos nada.

2.2 Despliegues: tres índices sobre los mismos objetos

Cada `Despliegue` (id, código, descripción, cuantizado y modalidad) se crea una sola vez y se guarda por referencia en tres lugares a la vez:

- `desplieguesPorId`: ABB sin comparador; `Despliegue` implementa `Comparable` por id.
- `desplieguesPorCodigo`: ABB con `ComparadorDesplieguePorCodigo`.
- `desplieguesPorModalidad`: array de 5 ABB, uno por modalidad, todos ordenados por código. Se accede con `modalidad.getIndice()`, que va de 0 a 4.

Como los tres índices apuntan al mismo objeto, no duplicamos datos: solo guardamos más referencias. Para buscar, creamos un despliegue "de búsqueda" que tiene cargada únicamente la clave (el id o el código). Por último, el `toString` de `Despliegue` ya arma el formato de salida que pide la letra, así que los listados solo tienen que ir concatenando.

¿Por qué un árbol por modalidad? La operación 8 pide O(k). Si tuviéramos un solo árbol por código y fuéramos filtrando por modalidad, igual habría que recorrer los n despliegues, o sea O(n). Separándolos, cada árbol contiene exactamente los k despliegues de esa modalidad y alcanza con recorrerlo. Java no permite crear un array genérico directamente, así que lo creamos así:

```
private ABB<Despliegue>[] crearArbolesPorModalidad() {
    ABB<Despliegue>[] arboles = new ABB[Modalidad.values().length];
    for (int i = 0; i < Modalidad.values().length; i++) {
        arboles[i] = new ABB<>(new ComparadorDesplieguePorCodigo());
    }
    return arboles;
}
```

2.3 Grafo de regiones

La red la modelamos con el grafo visto en clase: una matriz de adyacencia de `Arista` más un array de vértices. Como `maxRegiones` llega en `inicializarSistema`, sabemos de antemano el tamaño máximo, así que una matriz de M × M encaja bien. Además, saber si existe un enlace es un acceso directo a la matriz.

- `Region` (el vértice): código, nombre, país y zona. `equals` y `compareTo` comparan por código, y `toString` devuelve `codigo;nombre;pais;zona`.
- `Arista`: es la del curso, pero en vez de un único peso guarda **latencia** y **costo**, porque cada enlace tiene dos pesos distintos y hay que poder minimizar cualquiera de los dos.
- `Grafo`: `agregarVertice`, `agregarArista`, `existeVertice`, `existeArista`, `estaLleno` y `obtenerPos` (búsqueda lineal en el array comparando con `equals`), más los recorridos de las operaciones 11 a 13.

Como el grafo es no dirigido, el constructor (con `esDirigido = false`) crea un único objeto `Arista` por cada par y lo pone en las dos celdas simétricas de la matriz:

```
for (int i = 0; i < cantMaxVertices; i++) {
    for (int j = i; j < cantMaxVertices; j++) {
        Arista arista = new Arista();
        aristas[i][j] = arista;   // misma instancia en [i][j] y [j][i]
        aristas[j][i] = arista;
    }
}
```

Entonces, al registrar el enlace A → B queda automáticamente B → A, que es justo lo que pide la letra. Por eso, si después se intenta registrar B → A, se devuelve ERROR_6.

2.4 Estructuras auxiliares de los recorridos

- **Operación 11**: es el `bfsConTuplas` del curso. Usa la `Cola<T>` genérica con tuplas (posición, saltos) y un array `visitados` para no repetir regiones. En lugar de imprimir cada vértice, como en la versión de clase, lo insertamos en un `ABB<Region>` local, que las deja ordenadas por código; al final `listarAscendente()` devuelve el formato pedido.
- **Operaciones 12 y 13**: es el Dijkstra del curso, con los arrays `visitados`, `costos` ("infinito" = `Integer.MAX_VALUE`) y `vengo` (-1 = origen o no alcanzado). Es un único método que recibe qué peso sumar, latencia o costo, así no repetimos el algoritmo. La ruta se arma recorriendo `vengo` desde el destino hacia atrás, en forma recursiva para que quede en orden origen → destino, y el resultado se devuelve en una `Tupla<Integer, String>` (total, ruta).

3. Justificación de los órdenes

Una aclaración antes de empezar: en un ABB sin balancear, si los elementos llegan en un orden más o menos aleatorio, la altura promedio es O(log n). En el peor caso, por ejemplo si los ids se insertan ya ordenados, el árbol degenera en una lista y la altura pasa a ser O(n). Por eso la letra pide los órdenes **en promedio**, y es lo que justificamos acá.

#	Operación	Orden pedido	Orden obtenido
1	Inicializar sistema	Sin restricción	$O(M^2)$
2	Registrar despliegue	O(log n) promedio	O(log n) promedio
3	Buscar despliegue por id	O(log n) promedio	O(log n) promedio
4	Listar por id ascendente	O(n)	O(n)
5	Listar por id descendente	O(n)	O(n)
6	Buscar despliegue por código	O(log n) promedio	O(log n) promedio
7	Listar por código ascendente	O(n)	O(n)
8	Listar por modalidad	O(k)	O(k)
9	Registrar región	Sin restricción	O(M)
10	Registrar enlace	Sin restricción	O(M)
11	Red de regiones por saltos	Sin restricción	$O(M^2)$

#	Operación	Orden pedido	Orden obtenido
12	Ruta de menor latencia	Sin restricción	$O(M^2)$
13	Ruta de menor costo	Sin restricción	$O(M^2)$

3.1 Operaciones con restricción (despliegues)

Operación 2 – Registrar despliegue. Las validaciones son O(1): revisar que no haya campos vacíos y aplicar la expresión regular a un código de largo fijo (12 caracteres) no depende de n. Después hay dos búsquedas (por id, para el ERROR_3, y por código, para el ERROR_4) y tres inserciones (en el árbol por id, en el árbol por código y en el de su modalidad). Cada una de esas cinco operaciones baja por una sola rama, o sea, cuesta O(altura) = O(log n) en promedio; la del árbol de la modalidad es incluso O(log k) ≤ O(log n). En total son 5 · O(log n) = **O(log n)** en promedio.

Operaciones 3 y 6 – Buscar por id y por código. Es una única búsqueda en el árbol correspondiente, que baja por una sola rama: O(log n) en promedio. El contador de nodos visitados se va pasando como parámetro en la recursión, así que no agrega costo. Armar el texto con `toString` es O(1), porque no depende de n. Total: **O(log n)** en promedio.

Operaciones 4, 5 y 7 – Listados. El recorrido in-order (o su espejo, para el descendente) visita cada nodo exactamente una vez, y por cada nodo agrega su texto a un `StringBuilder`, lo que cuesta O(1) amortizado porque el largo de cada despliegue no depende de n. Total: **O(n)**. Usamos `StringBuilder` a propósito: si concatenáramos `String` con `+` en cada paso, cada concatenación copiaría todo lo acumulado hasta el momento y el costo total crecería en forma cuadrática con n.

Operación 8 – Listar por modalidad. Se llega al árbol de la modalidad en O(1) con `getIndice()` y se lo recorre in-order. Ese árbol tiene exactamente k nodos, así que el recorrido es **O(k)**, independientemente de cuántos despliegues haya de las otras modalidades.

3.2 Operaciones sin restricción (grafo)

La letra no pide órdenes para estas operaciones, pero igual los analizamos:

- **1 – Inicializar**: vaciar los árboles es O(1); lo que domina es crear el grafo, que reserva la matriz de M × M: **$O(M^2)$**.
- **9 – Registrar región**: `estaLleno` es O(1); `existeVertice` recorre el array de vértices, O(M); y buscar el lugar libre también es O(M). Total: **O(M)**.
- **10 – Registrar enlace**: hay que ubicar las dos regiones con `obtenerPos`, O(M) cada una; después, ver si el enlace existe y crearlo es un acceso a la matriz, O(1). Total: **O(M)**.
- **11 – Red por saltos**: gracias a `visitados`, cada región entra a la cola como máximo una vez, y por cada región que sale se recorre su fila de la matriz, O(M). Eso da $O(M^2)$. Además, cada región alcanzada se inserta en el ABB, O(log M) en promedio, lo que suma O(M log M). Total: **$O(M^2)$**.
- **12 y 13 – Rutas**: el ciclo principal de Dijkstra se ejecuta una vez por región. En cada vuelta se busca el no visitado de menor costo, O(M), y se recorre su fila de la matriz, O(M). Eso da $O(M^2)$. Armar la ruta con `vengo` pasa a lo sumo una vez por cada región, O(M). Total: **$O(M^2)$**.

4. Decisiones y supuestos

Hay algunos casos que la letra no deja del todo cerrados. Estas son las decisiones que tomamos:

- **Strings vacíos.** Tomamos como vacío un `null`, un `""` o un texto que tiene solo espacios (lo chequeamos con `trim()`), siguiendo el ejemplo de la letra, donde una descripción `"  "` da ERROR_1.
- **Orden de los errores.** Respetamos exactamente el orden de la tabla de cada operación. Por ejemplo, en la operación 9 se controla primero si el sistema está lleno, y en la 11 primero si la cantidad es negativa.
- **Modalidad nula en la operación 8.** La letra dice que no hay errores posibles, así que, si llega `null`, devolvemos OK con el listado vacío en lugar de lanzar una excepción.
- **Enlace de una región consigo misma.** La letra no define un error para este caso, así que lo aceptamos. No afecta al BFS ni a Dijkstra, porque esa región ya está marcada como visitada. Queda sujeto a lo que se aclare en el foro.
- **Empates en Dijkstra.** Si hay dos rutas con el mismo total, queda la que el algoritmo encuentra primero: entre los no visitados de menor costo se elige el de menor posición en el array, y un costo solo se actualiza si el nuevo es estrictamente menor. Es el mismo comportamiento que la versión del curso.
- **Reinicializar.** `inicializarSistema` vacía todos los árboles y crea un grafo nuevo, así que se puede llamar varias veces sobre la misma instancia (los tests provistos lo hacen).

5. Estrategia de testing

Armamos una clase de prueba JUnit 5 por operación (de `Test01` a `Test13`), además de ajustar las pruebas que venían con el proyecto. En cada clase buscamos cubrir tres cosas: el caso OK con el formato exacto de salida, cada uno de los errores, y la precedencia entre errores. Esto último quiere decir que, cuando una llamada cumple con dos errores a la vez, tiene que salir el que la letra pone primero.

También probamos casos borde. En la parte de despliegues:

- listados vacíos y con un solo elemento (sin `|` al final);
- orden lexicográfico real (`"DEP-10"` va antes que `"DEP-2"`, y las mayúsculas van antes que las minúsculas);
- árboles degenerados y conteo de nodos visitados en distintas ramas;
- que un registro fallido no quede guardado en ninguno de los índices, y que reinicializar borre todo.

En la parte del grafo:

- navegabilidad del enlace en ambos sentidos;
- ciclos y caminos alternativos sin regiones repetidas, componentes desconectadas y regiones aisladas;
- origen igual al destino, y casos sin ruta posible;

- una ruta directa más lenta que una indirecta, y casos donde la ruta de menor costo y la de menor latencia son distintas.

Además, reprodujimos los ejemplos trabajados en clase: el grafo de las notas de Dijkstra (de C a H da 12, pasando por C, A, D y H) y el del `MainDijkstra` (de 1 a 4 da 9).

Por último, corrimos también el set de pruebas que publicó la cátedra una semana antes de la entrega, para comparar contra nuestras propias pruebas y detectar desviaciones. Todas las pruebas se corren desde IntelliJ IDEA.

6. Uso de inteligencia artificial generativa

Siguiendo las pautas de la letra, declaramos el uso de herramientas de IA generativa en este trabajo:

- **Herramienta:** Claude (Anthropic), a través de claude.ai y Claude Code.
- **Contexto de uso:** análisis de la letra y consultas sobre la misma; revisión de los pull requests del repositorio contra la letra y el material del curso por si en la revisión humana se saltó algo crítico mediante skills básicas de la tecnología y generación de texto.